

Tout ce que tu dois savoir pour ta soutenance

CESIZen — Déployer et sécuriser les applications informatiques

Ce guide réunit en un seul document : ce qui a été fait sur le projet, comment réaliser la démonstration devant le jury, les concepts techniques à maîtriser, les 25 questions les plus probables avec leurs réponses, et les checklists de vérification avant la soutenance.

CONTENU DU GUIDE

1. **Ce qui a été fait** — récapitulatif complet des livrables du projet
2. **Tuto démonstration jury** — script pas-à-pas avec les mots à dire
3. **Les concepts techniques** — ce que tu dois savoir expliquer en 2 min
4. **Questions du jury** — 25 questions avec réponses préparées
5. **Checklists** — vérifications avant et pendant la soutenance

CHAPITRE 1

Ce qui a été fait sur le projet

Récapitulatif complet de tous les livrables — ce que tu peux montrer et revendiquer

Ce chapitre liste tout ce qui a été mis en place sur CESIZen dans le cadre du Bloc 3. Chaque élément est un point que tu peux montrer en démo ou revendiquer à l'oral. Garde ce chapitre en tête : c'est ta "liste de preuves".

1.1 Corrections de sécurité dans le code existant

Quatre problèmes de sécurité réels ont été identifiés dans le code et corrigés. Ce sont tes "quick wins" à l'oral : tu as fait une vraie analyse, pas que de la théorie.

#	Problème trouvé	Fichier modifié	Ce qui a été fait
R1	Brute force possible sur le login — aucune limite de tentatives	<code>routes/web.php</code> et <code>routes/api.php</code>	Ajout du middleware <code>throttle:5,1</code> sur les routes POST <code>/connexion</code> et <code>/api/login</code> → maximum 5 tentatives par minute par IP, puis blocage automatique
R2	XSS possible via les paramètres URL injectés dans du JavaScript	<code>app/Http/Controllers/BreathingController.php</code>	Cast <code>(int)</code> ajouté sur les 4 paramètres de respiration (inspiration, apnée, expiration, cycles) avant qu'ils soient injectés dans <code><script></code> — un entier ne peut pas contenir de code malveillant
R4	<code>APP_DEBUG=true</code> dans le template <code>.env.example</code> — risque d'afficher les erreurs en production	<code>.env.example</code>	Valeur changée à <code>false</code> — le template est maintenant sécurisé par défaut
R5	En-têtes HTTP de sécurité absents de toutes les réponses	<code>app/Http/Middleware/SecurityHeadersMiddleware.php</code> (nouveau fichier) + <code>bootstrap/app.php</code>	Nouveau middleware créé et enregistré sur toutes les routes web : X-Content-Type-Options (anti-sniffing), X-Frame-Options (anti-clickjacking), Referrer-Policy, Permissions-Policy (désactive micro/caméra/géoloc)

✅ **VALIDÉ** — Ces 4 correctifs sont dans le code — tu peux les montrer avec `git log` et ouvrir les fichiers en direct.

1.2 Configuration de l'environnement .gitignore

Le `.gitignore` a été audité et amélioré. Un fichier `android/.gitignore` a été créé pour exclure les builds Android (qui faisaient gonfler le dépôt et créaient du bruit dans les commits).

Fichier	Ajouts
<code>.gitignore</code>	Ajout de <code>/.claude</code> , <code>/index.php</code> , <code>android/local.properties</code> — le <code>.env</code> était déjà exclu
<code>android/.gitignore</code>	Crée : exclut <code>/app/build/</code> , <code>/.gradle/</code> , <code>/build/</code> , <code>*.iml</code> , <code>local.properties</code>

1.3 Docker — Environnement de test

Un environnement de test complet et reproductible en une commande a été créé.

Fichier créé	Rôle
<code>Dockerfile</code>	Image PHP 8.3 Apache + Node 20 + Composer + Vite build — reproductible sur n'importe quelle machine avec Docker
<code>docker-compose.yml</code>	Orchestre 2 conteneurs : app (port 8080) + db (MySQL 8, volume persistant) avec healthcheck
<code>docker/apache.conf</code>	Configuration Apache : DocumentRoot sur /public (obligatoire pour Laravel)
<code>docker/entrypoint.sh</code>	Script de démarrage : génère APP_KEY si absent, attend MySQL, lance les migrations
<code>.dockerignore</code>	Exclut vendor, node_modules, .env, .git du build — images plus légères
<code>Makefile</code>	Raccourcis : <code>make up</code> , <code>make seed</code> , <code>make shell</code> , <code>make reset</code>

 **JURY** — Tu peux lancer `make up` devant le jury et montrer l'application tourner sur localhost:8080 en moins de 2 minutes. C'est ton argument le plus fort pour l'environnement de test.

1.4 CI/CD — Pipeline GitHub Actions

Un pipeline automatique s'exécute à chaque push sur GitHub.

```
# Fichier : .github/workflows/ci.yml
# Déclenché sur : push main/develop, PR vers main

Job 1 – tests
  → PHP 8.3 + cache Composer
  → 44 tests Pest sur SQLite en mémoire
  → Durée : ~1 min 30

Job 2 – docker (only if: needs.tests.result == 'success')
  → docker build -t cesizen:{SHA} .
  → Durée : ~1 min
```

✓ **VALIDÉ** — Le pipeline bloque le build Docker si un seul test échoue. Aucun code cassé ne peut atteindre l'environnement de test.

1.5 Ticketing — GitHub Issues + Projects

Élément	Détail
Labels de type	<code>bug</code> · <code>evolution</code> · <code>maintenance</code> · <code>dependabot</code>
Labels de criticité	<code>bloquant-critique</code> · <code>bloquant-fort</code> · <code>majeur</code> · <code>mineur</code>
Labels de statut	<code>en-cours</code> · <code>en-test</code>
Template Bug report	<code>.github/ISSUE_TEMPLATE/bug_report.md</code> — pré-rempli avec cases à cocher de criticité
Template Feature request	<code>.github/ISSUE_TEMPLATE/feature_request.md</code> — critères d'acceptation, priorité
Template Pull Request	<code>.github/PULL_REQUEST_TEMPLATE.md</code> — checklist tests / sécurité / revue
Board Projects	4 colonnes : À traiter → En cours → En test → Terminé
Dependabot	<code>.github/dependabot.yml</code> — PRs auto hebdo Composer + npm, mensuelle GitHub Actions

1.6 Documents produits

Fichier	Contenu	Format
CESiZen_Dossier_Bloc3.pdf	Dossier technique ~20 pages : déploiement + maintenance + sécurité	PDF A4
CESiZen_Presentation_Bloc3.pdf	Présentation 15 slides pour la soutenance	PDF A4 paysage
script_demo.md	Script de démo jury avec les mots à dire	Markdown
fiche_jury.md	15 questions + réponses préparées	Markdown

⚠ **ATTENTION** — Le `.env` ne doit jamais apparaître dans git. Vérification : `git log -- .env` doit retourner zéro résultat. Si quelqu'un te pose la question, tu peux le prouver en direct.

CHAPITRE 2

Tuto démonstration jury

Script pas-à-pas avec le timing, les actions exactes et les phrases à dire

La démo dure environ 6 minutes dans une soutenance de 20 minutes. Elle se divise en deux parties : le CI/CD et le ticketing. Tu peux aussi proposer une démo Docker si le jury le demande.

🎓 **JURY** — Le jury veut voir que tu sais te servir de tes outils, pas juste en parler. Une démo même courte vaut plus qu'un long discours. Ouvre les vraies pages GitHub, ne montre pas des captures d'écran.

2.1 Prérequis — à préparer AVANT d'entrer dans la salle

- ☐ Ouvrir github.com/louisbeaujoin/cesizen dans un onglet — tu seras déjà connecté
- ☐ Avoir Docker Desktop démarré (icône visible dans la barre système) — pas encore `make up`
- ☐ Ouvrir un terminal PowerShell dans `C:\wamp64\www\cesizen`
- ☐ Ouvrir VS Code sur le projet (pour éventuellement montrer du code)
- ☐ Couper les notifications Windows et mettre le son en sourdine
- ☐ Avoir une connexion internet active (GitHub Actions se montre en ligne)
- ☐ Mettre en plein écran le navigateur — police de lecture $\geq 14\text{pt}$ (Ctrl+= pour zoomer)

2.2 Démo A — Le pipeline CI/CD (3 minutes)

1

Montrer l'historique des commits sur GitHub (30 sec)

Ouvre `github.com/louisbeaujoin/cesizen` → clique sur le nombre de commits (ex: "6 commits") → fais défiler la liste.

"Chaque commit correspond à une étape précise du projet. Je suis la convention Conventional Commits : feat pour une nouvelle fonctionnalité, fix pour un correctif, ci pour le pipeline. Ça permet de comprendre l'historique du projet d'un coup d'œil."

2

Montrer le pipeline Actions (1 min 30)

Onglet **Actions** → cliquer sur le dernier workflow réussi → montrer les 2 jobs.

"Ici on voit le Job 1 — les 44 tests Pest s'exécutent automatiquement sur une base SQLite en mémoire. Ça prend environ 1 minute 30. Le Job 2 — le build Docker — ne démarre qu'une fois les tests passés. Si un test échoue, le build Docker est bloqué. C'est le filet de sécurité."

3

Push en live pour déclencher le pipeline (1 min) — optionnel mais impressionnant

Dans VS Code, ouvrir n'importe quel contrôleur, ajouter un commentaire anodin, puis dans le terminal :

```
git add app/Http/Controllers/HomeController.php
git commit -m "docs: demo pipeline CI soutenance"
git push origin develop
```

Revenir immédiatement sur l'onglet Actions de GitHub → actualiser → le workflow apparaît en cours (cercle jaune).

"Le pipeline se déclenche en quelques secondes après le push. On voit le cercle jaune — les tests sont en cours. Une fois passés, le build Docker démarre automatiquement."

Si le jury te coupe et pose une question pendant la démo : réponds calmement, laisse le pipeline tourner en arrière-plan, reviens dessus après ta réponse. C'est encore plus impressionnant d'avoir le pipeline qui s'est terminé pendant que tu répondais.

2.3 Démo B — Le ticketing GitHub Projects (3 minutes)

1

Montrer le board (1 min)

Onglet **Projects** → ouvrir "CESIZen — Suivi du projet" → vue Board avec les 4 colonnes.

"Le board reflète l'état réel du projet. On voit immédiatement ce qui est en attente, ce qui est en cours de développement, ce qui attend une validation en test. Le Ministère peut consulter ce board à tout moment sans avoir besoin d'un outil séparé."

2

Ouvrir un ticket bug bloquant (1 min)

Aller dans **Issues** → filtrer par label `bloquant-critique` → ouvrir un ticket avec ce label.

"Quand le Ministère signale un incident, il utilise ce template pré-rempli. Il sélectionne la criticité — ici bloquant-critique — ce qui déclenche le SLA : prise en compte sous 1 heure, correction sous 3 heures. Le template garantit qu'on a toutes les informations pour reproduire le bug dès le premier message."

3

Montrer Dependabot (30 sec)

Dans Issues → filtrer par label `dependabot` → montrer une PR Dependabot fermée ou ouverte.

"Ce ticket a été créé automatiquement par Dependabot — un outil GitHub qui surveille les dépendances. Chaque lundi il scanne le projet et ouvre une Pull Request si une mise à jour est disponible. Les tests CI s'exécutent dessus, et je merge si tout est vert. La veille technologique est en grande partie automatisée."

4

Créer un ticket en live si le jury le demande (30 sec)

Issues → New Issue → choisir le template Bug report → montrer les champs pré-remplis.

"Je peux créer un nouveau ticket en moins de 30 secondes avec le template. Les cases à cocher de criticité sont déjà là, les étapes de reproduction, l'environnement. Rien à mémoriser pour le Ministère."

2.4 Démo C — Docker (si le jury le demande)

Cette démo prend 2-3 minutes. Ne la lance que si le jury demande à voir l'application tourner, ou si tu as du temps.

1

Lancer l'environnement

Dans le terminal PowerShell ouvert dans le projet :

```
make up  
# Attendre : "cesizen-app | Server started"
```

"Une seule commande suffit. Docker build l'image PHP 8.3 Apache, démarre la base de données MySQL, attends qu'elle soit prête, lance les migrations. Tout est automatique."

2

Charger les données de démo

```
make seed
```

3

Ouvrir dans le navigateur

Naviguer vers `http://localhost:8080` → montrer la page d'accueil, se connecter avec un compte de démo.

"L'environnement de test est identique à ce qui tournerait en production. N'importe qui dans l'équipe peut le lancer avec `make up`, sans avoir PHP, MySQL ou Node.js installés sur sa machine."

4

Arrêter proprement

```
make down
```

Si Docker plante ou est lent : ne panique pas. Dis "L'image est en cours de build, ça prend 1-2 minutes la première fois car Node.js compile les assets. En usage normal l'image est déjà construite." et montre autre chose en attendant.

2.5 Montrer les correctifs de sécurité dans le code

Si le jury pose des questions sur la sécurité, tu peux ouvrir VS Code et montrer le code réel.

Ce qu'ils demandent	Fichier à ouvrir	Ligne à montrer
"Montre-moi le rate limiting"	<code>routes/web.php</code>	La ligne avec <code>throttle:5,1</code> sur <code>Route::post('/connexion')</code>
"Montre-moi la protection XSS"	<code>app/Http/Controllers/BreathingController.php</code>	Les 4 lignes avec <code>(int)</code> <code>\$request->input(...)</code>
"Montre-moi les en-têtes HTTP"	<code>app/Http/Middleware/SecurityHeadersMiddleware.php</code>	Les 4 <code>\$response->headers->set(...)</code>
"Comment Blade protège contre le XSS ?"	N'importe quelle vue dans <code>resources/views/</code>	Montrer <code>{{ \$variable }}</code> vs <code>{!! \$variable !!}</code>
"Montre-moi que .env n'est pas dans git"	Terminal	<code>git log -- .env</code> → aucun résultat

CHAPITRE 3

Les concepts techniques à maîtriser

Ce que tu dois pouvoir expliquer en 2-3 phrases claires, sans hésitation

Le jury ne teste pas ta capacité à réciter une définition Wikipedia. Il teste ta compréhension : est-ce que tu sais *pourquoi* tu as fait ce choix, et est-ce que tu peux l'expliquer simplement à quelqu'un qui ne connaît pas le sujet.

📌 **JURY** — La règle d'or : si tu ne peux pas l'expliquer en 2 phrases sans notes, tu ne le maîtrises pas encore. Entraîne-toi à dire ces définitions à voix haute.

3.1 Docker et les conteneurs

C'est quoi ?

Docker permet d'emballer une application avec tout ce dont elle a besoin (PHP, Apache, MySQL...) dans des boîtes isolées appelées **conteneurs**. Ces boîtes peuvent s'exécuter sur n'importe quelle machine de manière identique.

Différence avec une VM

Une VM émule un ordinateur entier (slow). Un conteneur partage le noyau Linux de la machine hôte — il démarre en secondes et consomme bien moins de ressources.

Pourquoi sur ce projet ?

Évite le "ça marche chez moi". L'environnement de test est identique sur toutes les machines. `make up` suffit, pas besoin d'installer PHP ou MySQL manuellement.

Vocabulaire clé

- **Image** : recette de cuisine (le Dockerfile)
- **Conteneur** : le plat préparé à partir de la recette
- **Volume** : disque dur persistant (les données survivent à l'arrêt)
- **docker-compose** : orchestre plusieurs conteneurs ensemble

Les commandes Docker à connaître par cœur

```
make up          # Build L'image + démarre app + db
make seed        # Migrations + données de démo (seeders)
make logs        # Voir Les logs en temps réel
make shell       # Ouvrir un terminal dans le conteneur app
make down        # Arrêter et supprimer les conteneurs
make reset       # Supprime les données + repart de zéro

# Équivalents Docker directs :
docker ps        # Lister les conteneurs en cours
docker compose up -d --build # Ce que make up fait vraiment
docker compose down -v      # Arrêt + suppression des volumes
```

3.2 GitHub Actions et la CI/CD

CI = Intégration Continue

À chaque push de code, un serveur automatique exécute les tests. Si les tests passent, on sait que le code ne casse rien. Si ça échoue, le développeur est notifié immédiatement.

CD = Livraison Continue

Une fois les tests passés, l'application est automatiquement packagée (image Docker) et prête à être déployée. Dans ce projet, le CD s'arrête au build de l'image — le déploiement prod reste manuel.

Comment fonctionne le fichier .github/workflows/ci.yml

```
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  tests:                                # Job 1
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: shivammathur/setup-php@v2
        with: { php-version: '8.3' }
      - run: composer install
      - run: composer test              # 44 tests Pest, SQLite en mémoire

  docker:                               # Job 2
    needs: tests                        # ← bloquant : ne démarre que si Job 1 réussit
    runs-on: ubuntu-latest
    steps:
      - run: docker build -t cesizen:${{ github.sha }} .
```

👉 **JURY** — Question piège fréquente : "Pourquoi tu utilises SQLite dans les tests et MySQL en prod ?" → Réponse : SQLite tourne en mémoire vive, donc les tests démarrent en millisecondes sans avoir à lancer un vrai serveur MySQL. Pour les tests unitaires et fonctionnels de Laravel (pas de requêtes SQL complexes spécifiques à MySQL), c'est suffisant et bien plus rapide. Laravel abstrait les différences via Eloquent.

3.3 Git Flow et conventions

Le Git Flow simplifié utilisé sur ce projet

```
# Structure des branches :
main          ← code stable, jamais développé directement dessus
develop       ← branche d'intégration continue
feature/xyz   ← une fonctionnalité, créée depuis develop, mergée dans develop

# Cycle d'une fonctionnalité :
git checkout develop
git checkout -b feature/add-export-csv    # Créer la branche
# ... développement ...
git commit -m "feat(export): export CSV des sessions de respiration"
git push origin feature/add-export-csv
# → Ouvrir une Pull Request develop ← feature/add-export-csv
# → Tests CI s'exécutent
# → Review de code
# → Merge dans develop
```

Conventional Commits — les types à connaître

Type	Usage	Impact SemVer
<code>feat</code>	Nouvelle fonctionnalité	v1.x.0 → incrémente MINEUR
<code>fix</code>	Correction de bug	v1.0.x → incrémente PATCH
<code>ci</code>	Modification du pipeline CI/CD	Pas d'impact SemVer
<code>chore</code>	Maintenance, mise à jour de dépendances	Pas d'impact SemVer
<code>docs</code>	Documentation uniquement	Pas d'impact SemVer
<code>refactor</code>	Remaniement du code sans changer le comportement	Pas d'impact SemVer
<code>BREAKING CHANGE</code>	Changement incompatible avec l'ancienne version	v2.0.0 → incrémente MAJEUR

Semantic Versioning — comment lire un numéro de version

```
1 . 2 . 3
↑   ↑   ↑
MAJEUR MINEUR CORRECTIF
```

MAJEUR : changement incompatible (breaking change)

MINEUR : nouvelle fonctionnalité rétrocompatible → feat

CORRECTIF : correction de bug rétrocompatible → fix

3.4 OWASP Top 10 — les 10 vulnérabilités à connaître

L'OWASP (Open Web Application Security Project) publie la liste des 10 vulnérabilités web les plus courantes. C'est la référence mondiale. On t'en demandera sûrement 3-4 en détail.

Référence	Nom	Ce que c'est en simple	Sur CESIZen
A01:2021	Broken Access Control	Un utilisateur accède à des ressources qu'il ne devrait pas voir (ex: voir le profil d'un autre utilisateur)	NATIF LARAVEL Double middleware <code>auth + admin</code> sur les routes admin
A02:2021	Cryptographic Failures	Données sensibles non chiffrées ou mal chiffrées (mots de passe en clair, HTTPS absent)	NATIF LARAVEL Cast <code>'hashed'</code> sur User → bcrypt 12 rounds automatique
A03:2021	Injection	Code malveillant injecté dans une requête SQL ou un script JavaScript (SQLi, XSS)	NATIF + AJOUTÉ Eloquent ORM (SQL), Blade (XSS HTML), cast <code>(int)</code> (XSS JS)
A04:2021	Insecure Design	Architecture logicielle non pensée pour la sécurité dès la conception	PRÉVENU MVC Laravel + séparation des rôles dès le début
A05:2021	Security Misconfiguration	Mauvaise configuration : DEBUG=true en prod, en-têtes HTTP absents, droits trop larges	CORRIGÉ APP_DEBUG=false + SecurityHeadersMiddleware
A06:2021	Vulnerable Components	Utilisation de bibliothèques avec des failles connues et non mises à jour	AUTOMATISÉ Dependabot → PRs auto chaque semaine
A07:2021	Auth & Session Failures	Sessions mal gérées, pas de limite de tentatives de connexion, tokens non invalidés	CORRIGÉ throttle:5,1 + session()->regenerate() post-login
A08:2021	Software Integrity Failures	Code ou dépendances non vérifiés (supply chain attacks, mises à jour sans vérification)	COMPOSER.LOCK Versions figées + hash vérifiés par Composer
A09:2021	Logging & Monitoring	Absence de logs → impossible de détecter une attaque	PARTIEL Logs Laravel natifs, monitoring prod à ajouter (Sentry)
A10:2021	SSRF	L'application fait des requêtes vers des serveurs internes à la demande d'un attaquant	N/A CESIZen ne fait pas de requêtes serveur-à-serveur

3.5 Les 4 correctifs expliqués en détail

Rate Limiting (Brute Force — R1)

```
# routes/web.php
Route::post('/connexion', [AuthController::class, 'login'])
    ->middleware('throttle:5,1');
#           ↑  ↑
#           5 requêtes max par 1 minute par IP
```

En simple : sans ça, un attaquant peut écrire un script qui teste 10 000 mots de passe en quelques secondes. Avec `throttle:5,1`, après 5 tentatives Laravel retourne une erreur HTTP 429 (Too Many Requests) et bloque pendant 1

minute. Ça rendrait une attaque brute force impraticable — il faudrait des années.

Cast (int) contre XSS dans du JavaScript (R2)

```
# app/Http/Controllers/BreathingController.php
# AVANT (dangereux) :
$inspiration = $request->input('inspiration', 5);

# APRÈS (sécurisé) :
$inspiration = (int) $request->input('inspiration', 5);

# Dans la vue Blade, cette valeur est injectée dans du JS :
let inspiration = {{ $inspiration }}; // Maintenant forcément un entier
```

En simple : si un attaquant envoie `?inspiration=5;alert('XSS')` dans l'URL, sans le cast PHP transformerait ça en code JavaScript malveillant. Le `(int)` convertit n'importe quelle valeur en entier — un entier ne peut pas contenir de code, donc l'attaque échoue.

Pourquoi Blade ne suffit pas ici ? Blade's `{{ }}` protège l'HTML (il encode `<` en `<`), mais dans un contexte JavaScript, les caractères dangereux sont différents (`;`, `)`). Le cast `(int)` est la bonne solution pour des valeurs qui doivent être des nombres.

SecurityHeadersMiddleware (R5)

```
# app/Http/Middleware/SecurityHeadersMiddleware.php
X-Content-Type-Options: nosniff
→ Empêche le navigateur de "deviner" le type d'un fichier
Ex: un .jpg contenant du JS ne sera pas exécuté comme JS

X-Frame-Options: SAMEORIGIN
→ Interdit d'intégrer CESIZen dans une iframe d'un autre site
Protège contre le clickjacking (clic sur un bouton invisible)

Referrer-Policy: strict-origin-when-cross-origin
→ Limite les infos envoyées quand on clique sur un lien externe
L'URL complète n'est pas transmise aux sites tiers

Permissions-Policy: geolocation=(), microphone=(), camera=()
→ Désactive explicitement la géoloc, le micro et la caméra
Un script malveillant ne peut pas demander ces permissions
```

APP_DEBUG=false (R4)

En simple : quand `APP_DEBUG=true`, Laravel affiche les erreurs complètes dans le navigateur : le code source, les variables, la structure de la base de données, la configuration. C'est indispensable en développement, mais catastrophique en production — un attaquant peut obtenir un plan complet de l'application en provoquant une erreur. La valeur doit être `false` en production, et le template `.env.example` doit refléter ça.

3.6 RGPD — Ce que tu dois savoir

Concept	Explication simple
Qu'est-ce que le RGPD ?	Le Règlement Général sur la Protection des Données — une loi européenne de 2018 qui encadre la collecte et le traitement des données personnelles. Toute organisation qui traite des données de citoyens européens doit s'y conformer.
Minimisation des données	On ne collecte que ce qui est strictement nécessaire. Sur CESIZen : nom, email, historique de respiration. Pas de numéro de téléphone, pas de date de naissance, pas de localisation.
Durées de conservation	Les données ne doivent pas être gardées indéfiniment. Sur CESIZen : sessions de respiration → 2 ans glissants, sessions de connexion → 120 minutes, compte → durée du compte + 1 an après suppression.
Droits des utilisateurs	Accès (voir ses données), Rectification (les corriger), Effacement (les supprimer), Portabilité (les exporter). Sur CESIZen : accès et rectification via /profil, effacement par l'admin, portabilité prévue en v1.1.0.
Notification CNIL 72h	En cas de violation de données (fuite, piratage) qui présente un risque pour les personnes, le DPO doit notifier la CNIL dans les 72 heures. C'est une obligation légale, pas une option.
DPO	Délégué à la Protection des Données — responsable RGPD dans l'organisation. Sur ce projet, c'est côté Ministère.
Données de santé	Les sessions de respiration pourraient être qualifiées de données de santé (Art. 9 RGPD — catégorie spéciale). En production réelle, un hébergement certifié HDS (Hébergeur de Données de Santé) serait requis.

3.7 Les SLA expliqués

SLA = Service Level Agreement = Accord de Niveau de Service. C'est la partie du contrat qui définit les délais de réponse que le prestataire s'engage à respecter selon la gravité du problème.

Les 4 niveaux de criticité :

Bloquant critique → Prise en compte : 1h | Correction : 3h
 Ex: Application inaccessible, fuite de données
 Action : traitement immédiat, tout le reste s'arrête

Bloquant fort → Prise en compte : 2h | Correction : 6h
 Ex: Connexion impossible pour tous les utilisateurs
 Action : haute priorité, développement dès que possible

Majeur → Prise en compte : 7h | Correction : 16h (~2j)
 Ex: Affichage incorrect sur mobile, fonctionnalité dégradée
 Action : traitement dans la semaine

Mineur → Prise en compte : 1 jour | Correction : 40h (~5j)
 Ex: Faute d'orthographe, coquille CSS
 Action : regroupé dans le prochain lot de maintenance

CHAPITRE 4

25 questions du jury avec réponses

Les questions les plus probables, préparées, dans les mots que tu peux dire à l'oral

📌 **JURY** — Lis ces réponses à voix haute plusieurs fois. L'objectif n'est pas de les mémoriser mot pour mot — c'est d'arriver à en sortir l'essentiel en 30 secondes même sous pression.

4.1 Questions sur le déploiement

Q : Pourquoi avoir choisi Docker pour l'environnement de test et pas juste WAMP ?

R : Docker garantit que l'environnement est identique sur toutes les machines. Avec WAMP, l'environnement est propre à ma machine Windows — quelqu'un d'autre qui clone le projet peut avoir une version PHP différente, une config MySQL différente. Docker élimine ce risque : une commande `make up` donne exactement le même environnement partout, en quelques minutes, sans rien installer.

Mot-clé à placer : "reproductible"

Q : C'est quoi la différence entre un Dockerfile et un docker-compose.yml ?

R : Le Dockerfile décrit comment construire une image — c'est la recette : "pars de PHP 8.3, installe Node.js, ajoute Composer, copie le code, exécute les commandes de build". Le docker-compose.yml orchestre plusieurs images ensemble — il dit "lance un conteneur app depuis l'image CESIZen sur le port 8080, lance un conteneur db depuis l'image MySQL 8, connecte-les". Un seul projet, deux rôles distincts.

Q : Pourquoi GitHub Actions et pas Jenkins ou GitLab CI ?

R : GitHub Actions est nativement intégré dans GitHub — zéro configuration externe, zéro serveur à maintenir. Comme j'utilise déjà GitHub pour le code, les issues et le board de suivi, tout est centralisé au même endroit. C'est aussi gratuit pour les dépôts publics. Jenkins nécessiterait un serveur dédié à maintenir. C'est le principe de simplicité : un seul outil pour tout.

Q : Comment tu déploies en production dans la vraie vie ?

R : En production, ce serait un VPS Linux Debian avec Nginx + PHP-FPM + MySQL. On clonerait le dépôt, on configurerait le .env avec les vraies clés, on lancerait `php artisan migrate --force` et `php artisan config:cache` pour optimiser. Le certificat SSL serait géré par Let's Encrypt avec certbot. Dans ce projet, la production est décrite architecturalement — l'environnement réellement opérationnel est l'environnement de test Docker.

Être honnête sur ce qui est théorique est valorisé par le jury.

Q : C'est quoi le Semantic Versioning et pourquoi c'est utile ?

R : C'est une convention de numérotation : MAJEUR.MINEUR.CORRECTIF. La version 1.2.3 signifie : 1 changement incompatible depuis le début, 2 nouvelles fonctionnalités ajoutées, 3 bugs corrigés. L'utilité : un développeur qui voit passer de v1.2.3 à v1.2.4 sait que c'est juste un bugfix, pas besoin de re-tester toute l'application. De v1.2.3 à v2.0.0, il sait qu'il y a un breaking change et qu'il faut revoir son intégration.

Q : C'est quoi l'intérêt du Makefile ?

R : C'est un raccourci pour éviter de mémoriser et taper des commandes Docker longues et complexes. `make up` remplace `docker compose up -d --build`. Ça rend le projet accessible à quelqu'un qui ne connaît pas Docker — il lit le Makefile, il voit les commandes disponibles. C'est aussi un contrat implicite avec l'équipe : les commandes standards du projet sont `make up`, `make seed`, `make test`.

4.2 Questions sur la maintenance

Q : Pourquoi GitHub Issues plutôt que Jira ou Trello ?

R : Parce que ça centralise tout dans un seul outil : le code, le CI/CD, les issues et le board de suivi sont tous sur GitHub. Avec Jira, on aurait un outil de plus à maintenir, à synchroniser, à facturer. Sur ce projet de formation, la simplicité prime. Et l'avantage concret est la traçabilité : je peux fermer un ticket automatiquement avec `Closes #13` dans un message de commit — le lien entre le problème et sa solution est permanent.

Q : Comment tu gères les SLA en pratique au quotidien ?

R : Dès qu'un ticket arrive, je le qualifie en criticité dans l'heure. Pour un bloquant critique, je commence la correction immédiatement — tout le reste s'arrête. Pour un majeur, je l'intègre dans la planification de la semaine. Pour les mineurs, je les regroupe en lots hebdomadaires pour optimiser le temps de développement. La clé est de qualifier rapidement — le SLA commence dès que le ticket est reçu, pas quand je commence à travailler dessus.

Q : C'est quoi Dependabot et comment ça fonctionne ?

R : Dependabot est un outil GitHub qui surveille automatiquement les dépendances du projet. Chaque lundi il scanne le `composer.json` pour PHP et le `package.json` pour JavaScript. Si une version plus récente existe, il ouvre automatiquement une Pull Request avec la mise à jour. Les tests CI s'exécutent dessus automatiquement. Si tout est vert, je merge. C'est de la veille technologique automatisée — je ne rate plus une mise à jour de sécurité critique.

Q : Comment tu communicates avec le Ministère en cas de problème ?

R : Pour les incidents bloquants critiques, un email direct au DSI et au DPO — pas d'attente de la consultation du board GitHub. Pour tout le reste, les tickets GitHub Issues sont le canal principal — traçabilité complète et historique permanent. Un compte-rendu mensuel est généré depuis le board Projects : tickets ouverts, fermés, délais respectés. Et une réunion mensuelle pour prioriser les évolutions à venir.

4.3 Questions sur la sécurité

Q : Qu'est-ce que l'OWASP Top 10 et pourquoi c'est la référence ?

R : L'OWASP — Open Web Application Security Project — est une fondation internationale à but non lucratif qui publie des ressources sur la sécurité web. Le Top 10 est leur liste des 10 catégories de vulnérabilités les plus critiques, mise à jour tous les 4-5 ans, basée sur des données réelles de milliers d'applications auditées dans le monde. C'est la référence car c'est empirique — ce sont les vrais problèmes qui arrivent dans les vraies applications, pas des théories.

Q : Comment Laravel protège contre l'injection SQL ?

R : Laravel utilise Eloquent, un ORM qui génère des requêtes préparées avec paramétrage. Concrètement, les valeurs utilisateur sont toujours séparées du code SQL — elles sont envoyées au moteur de base de données comme des paramètres distincts, jamais concaténées dans la chaîne SQL. Même si un utilisateur entre `' OR 1=1 --`, MySQL reçoit ça comme une valeur littérale dans la colonne email, pas comme du code SQL à exécuter. Il n'y a aucune requête SQL brute dans le code CESIZen.

Q : C'est quoi le CSRF et comment Laravel le protège ?

R : Le CSRF — Cross-Site Request Forgery — c'est quand un site malveillant fait soumettre un formulaire à ta place sur un site où tu es connecté. Par exemple un site qui charge silencieusement `cesizen.fr/supprimer-compte` dans une iframe. Laravel protège contre ça avec un token aléatoire généré à chaque session : `@csrf` dans les formulaires. Ce token est vérifié à chaque requête POST — un site tiers ne peut pas connaître ce token, donc sa requête est rejetée.

Q : Pourquoi tu as mis `throttle:5,1` et pas `throttle:10,1` ou `throttle:3,1` ?

R : C'est un compromis entre sécurité et expérience utilisateur. 5 tentatives par minute permet à quelqu'un qui se trompe de mot de passe de réessayer rapidement sans être bloqué de manière frustrante. En même temps, 5 tentatives par minute c'est 7 200 par jour — ça reste très lent pour une attaque brute force sur un mot de passe fort. En production, j'aurais aussi ajouté un blocage exponentiel (30 sec, 2 min, 10 min...) et éventuellement une validation CAPTCHA après plusieurs échecs.

Montrer que tu as réfléchi au compromis est plus valorisé que la valeur elle-même.

Q : Pourquoi tu n'as pas mis un Content Security Policy (CSP) ?

R : Le CSP est l'en-tête HTTP le plus puissant contre le XSS, mais aussi le plus complexe à configurer. Sur CESIZen, Tailwind CSS et Vite génèrent des styles inline, Alpine.js exécute du JavaScript inline — un CSP strict casserait l'application. J'aurais besoin de nonces cryptographiques ou de hashes par script. J'ai fait le choix de prioriser les 4 correctifs qui ne cassent rien et couvrent les risques réels identifiés. Le CSP est documenté comme amélioration future.

Ne pas avoir fait quelque chose n'est pas grave si tu sais pourquoi et si tu peux nommer la prochaine étape.

Q : Comment tu sais que le `.env` ne s'est jamais retrouvé dans git ?

R : Deux garanties. D'abord, le `.gitignore` exclut `.env` depuis la première ligne du projet — Git ne propose même pas de le stager. Ensuite, je peux le prouver en direct : `git log -- .env` retourne zéro commit — le fichier n'a jamais été suivi par git. Et en dernier recours, `git secret scan` ou `trufflehog` peuvent scanner l'historique complet à la recherche de secrets.

4.4 Questions sur le RGPD

Q : Qu'est-ce que tu as mis en place concrètement pour le RGPD ?

R : Trois choses concrètes. Premièrement, la minimisation : je ne collecte que nom, email et historique de sessions — rien de plus. Deuxièmement, des durées de conservation définies : 2 ans pour les sessions de respiration, compte + 1 an après suppression. Troisièmement, les droits d'accès et de rectification via la page profil. Ce qui manque et est prévu en v1.1.0 : l'export des données (droit à la portabilité de l'article 20).

Q : Que se passe-t-il si CESIZen subit une fuite de données ?

R : Il y a un plan de crise en 7 étapes. D'abord, qualifier la gravité et le périmètre. Ensuite, notifier le DSI et le DPO du Ministère dans les 2 heures. Mettre le serveur hors ligne si nécessaire pour stopper la fuite. La CNIL doit être notifiée dans les 72 heures sur [notifications.cnil.fr](https://www.cnil.fr/fr/notifications) — c'est une obligation légale avec risque d'amende allant jusqu'à 4 % du chiffre d'affaires mondial. Si les utilisateurs sont à risque élevé, ils doivent être informés par email. Enfin, une analyse post-incident pour éviter que ça se reproduise.

Q : C'est quoi la différence entre un DPO et un RSSI ?

R : Le DPO — Délégué à la Protection des Données — est responsable de la conformité RGPD : droits des personnes, durées de conservation, notifications CNIL. C'est un rôle juridique et organisationnel. Le RSSI — Responsable de la Sécurité des Systèmes d'Information — est responsable de la sécurité technique : firewall, sauvegardes, gestion des incidents, tests de pénétration. Les deux collaborent en cas d'incident de sécurité qui implique des données personnelles.

4.5 Questions générales et pièges

Q : Qu'est-ce que tu ferais différemment si c'était un vrai projet de production ?

R : Trois choses principalement. D'abord, le déploiement continu automatisé vers la production via GitHub Actions avec une approbation manuelle avant le push final. Ensuite, un monitoring applicatif : Sentry pour capturer les erreurs en temps réel et Uptime Robot pour être alerté si le site tombe. Enfin, un hébergement certifié HDS si les données de respiration sont qualifiées de données de santé au sens de l'Article 9 RGPD. Et le hachage des tokens API qui est actuellement stocké en clair.

Q : Quel est le coût de ton architecture en production ?

R : L'environnement DEV et TEST sont gratuits — WAMP local et Docker local. GitHub et GitHub Actions sont gratuits pour les projets publics. En production, un VPS OVHcloud Starter coûte environ 6 € par mois, le nom de domaine cesizen.fr environ 12 €/an, et Let's Encrypt pour le HTTPS est gratuit. Le total annuel de l'infrastructure serait de l'ordre de 80-90 € — largement dans l'enveloppe des 75 000 € du projet.

Q : Quelle est la différence entre tests unitaires et tests fonctionnels ?

R : Un test unitaire teste une seule fonction ou méthode en isolation — sans base de données, sans requête HTTP. Un test fonctionnel (ou d'intégration) teste une fonctionnalité complète : il simule une requête HTTP, passe par le middleware, le contrôleur, le modèle, la base de données, et vérifie la réponse finale. Sur CESIZen, les 44 tests Pest sont majoritairement des tests fonctionnels — ils vérifient que les routes retournent les bons statuts HTTP et que les données en base sont correctes.

Q : Pourquoi les tests utilisent SQLite en mémoire et pas MySQL ?

R : SQLite en mémoire démarre en millisecondes — aucun serveur à lancer, aucune connexion réseau. Les 44 tests s'exécutent en environ 2 secondes. Avec MySQL, il faudrait démarrer un service, créer une base de test, nettoyer entre chaque test — on serait plutôt à 10-15 secondes. Dans un pipeline CI qui s'exécute 20 fois par jour, ça compte. Laravel abstrait les différences via Eloquent, donc les requêtes fonctionnent identiquement sur les deux moteurs pour nos cas d'usage.

Q : Pourquoi avoir choisi Laravel plutôt que Symfony ou un framework JavaScript ?

R : Laravel est le framework PHP le plus utilisé au monde avec une communauté massive, une documentation exhaustive et un écosystème très riche. Il intègre nativement les protections de sécurité dont on a parlé (CSRF, XSS, SQL), un ORM puissant, un système d'authentification prêt à l'emploi. Pour un projet Ministère avec un budget défini et un délai de 12 mois, c'est le choix qui minimise les risques et maximise la productivité. C'est aussi ce que la demande du Ministère spécifiait.

CHAPITRE 5

Checklists et aide-mémoire

Ce qu'il faut vérifier avant, pendant et après la soutenance

5.1 Checklist — La veille de la soutenance

- ☐ Ouvrir `github.com/louisbeaujoin/cesizen` → vérifier que les commits et le pipeline sont visibles
- ☐ Vérifier que le dernier pipeline CI est vert (cercle vert dans l'onglet Actions)
- ☐ Vérifier que le board GitHub Projects a des tickets dans les bonnes colonnes
- ☐ Lancer `make up` une fois → vérifier que localhost:8080 fonctionne
- ☐ Lancer `make seed` → se connecter avec les comptes de démo, vérifier les pages
- ☐ Lancer `make down` → tu redémarras devant le jury si besoin
- ☐ Vérifier que Docker Desktop démarre bien (icône dans la barre)
- ☐ Ouvrir VS Code → vérifier que les fichiers clés sont accessibles rapidement
- ☐ Relire ce guide — en particulier les questions/réponses du Chapitre 4
- ☐ Préparer le diaporama : ouvrir `CESIZen_Presentation_Bloc3.pdf` en mode présentation

5.2 Checklist — Le jour J, avant d'entrer dans la salle

- ☐ Navigateur ouvert sur `github.com/louisbeaujoin/cesizen` — connexion vérifiée
- ☐ Docker Desktop démarré (mais pas `make up` encore)
- ☐ Terminal PowerShell ouvert dans `C:\wamp64\www\cesizen`
- ☐ VS Code ouvert sur le projet
- ☐ Diaporama PDF ouvert en plein écran (ou prêt à l'être)
- ☐ Ce guide ouvert en PDF (en cas de blanc, tu peux discrètement le consulter)
- ☐ Notifications Windows coupées, son en sourdine
- ☐ Connexion internet vérifiée

☐ Avoir mangé, bu de l'eau, respiré

5.3 Commandes à connaître par cœur

```
# — DOCKER —
make up           # Démarrer l'environnement de test
make seed         # Charger les données de démo
make down         # Arrêter proprement
make shell        # Terminal dans le conteneur
make reset        # Repartir de zéro

# — GIT —
git log --oneline  # Historique des commits (court)
git log -- .env    # Prouver que .env n'a jamais été commité
git status         # État du dépôt local

# — LARAVEL —
composer test      # Lancer les 44 tests Pest
php artisan route:list  # Voir toutes les routes et leurs middlewares
php artisan migrate:status  # État des migrations
```

5.4 Les fichiers clés à connaître

Fichier	Ce qu'il contient à montrer
routes/web.php	Le <code>throttle:5,1</code> sur le login
routes/api.php	Le <code>throttle:5,1</code> sur l'API login
app/Http/Controllers/BreathingController.php	Les cast <code>(int)</code> anti-XSS
app/Http/Middleware/SecurityHeadersMiddleware.php	Les 4 en-têtes HTTP de sécurité
bootstrap/app.php	L'enregistrement du SecurityHeadersMiddleware
.env.example	APP_DEBUG=false, toutes les variables sans valeurs réelles
.gitignore	La ligne <code>.env</code> qui exclut les secrets
Dockerfile	La recette de l'image Docker
docker-compose.yml	Les 2 services app + db
Makefile	Les commandes simplifiées
.github/workflows/ci.yml	Le pipeline : 2 jobs, le <code>needs: tests</code>
.github/dependabot.yml	La veille automatique

5.5 Les chiffres à retenir pour impressionner

Chiffre	Ce que ça représente
44	Tests automatisés Pest — tous passants
83 %	Risques OWASP couverts (objectif 75 % dépassé)
4	Correctifs de sécurité appliqués dans le code réel
3	Environnements définis (DEV, TEST, PROD)
~2 min 30	Durée totale du pipeline CI/CD
1h / 3h	SLA bloquant critique : prise en compte / correction
72h	Délai légal CNIL en cas de fuite de données
6	Points OWASP couverts nativement par Laravel
75 000 €	Budget du projet Ministère
12	Mois de durée du projet

5.6 Gestion du stress et des moments difficiles

👤 **JURY** — Si tu ne sais pas répondre à une question : "C'est une bonne question, je n'ai pas implémenté ça dans ce projet. Ce que j'aurais fait c'est [logique approximative]. Est-ce que c'est le sens de votre question ?" Le jury préfère quelqu'un qui raisonne à quelqu'un qui invente.

Si la démo plante : garde ton calme. Dis "L'environnement a un problème — dans un contexte professionnel j'aurais une démo de secours. Je vais vous montrer le code à la place." Montrer du code avec assurance récupère la situation.

Si on te demande quelque chose hors périmètre (Android, base de données avancée...) : "Ça dépasse le périmètre de ce Bloc 3 qui se concentre sur le déploiement et la sécurisation. Je l'ai pris en compte en excluant les builds Android du pipeline CI, mais l'application Android elle-même n'est pas dans le scope de cette soutenance."

Structure magique pour toute réponse : QUOI → POURQUOI → COMMENT.

"J'ai ajouté [QUOI] parce que [POURQUOI], et ça fonctionne en [COMMENT]."

Ex : "J'ai ajouté le middleware throttle:5,1 parce que sans limitation un attaquant peut tester des milliers de mots de passe, et ça fonctionne en comptant les requêtes par IP et en retournant un HTTP 429 après 5 tentatives."

✅ **VALIDÉ** — Tu as travaillé sur un vrai projet avec de vrais outils. Docker tourne, le pipeline CI est vert, les tickets existent, les correctifs sont dans le code. Tu peux tout montrer en live. C'est ça qui compte.